

[Blog](#) [Archive](#) [Tags](#) [Projects](#)

Sean McGary

Software Engineer, builder of webapps

How to build a fault-tolerant redis cluster with sentinel

by Sean McGary on Jul 09, 2015

Today, I'm going to show you how to setup a fault-tolerant master/slave redis cluster using sentinel to failover lost nodes.

[Redis](#) is a very versatile database, but what if you want to run it on a cluster? A lot of times, people will run redis as a standalone server with no backup. But what happens when that machine goes down? Or what if we want to migrate our redis instance to a new machine without downtime?

All of this is possible by creating a replica set (master node and n many slave nodes) and letting [sentinel](#) watch and manage them. If sentinel discovers that a node has disappeared, it will attempt to elect a new master node, provided that a majority of sentinels in the cluster agree (i.e. quorum).

The quorum is the number of Sentinels that need to agree about the fact the master is not reachable, in order for really mark the slave as failing, and eventually start a fail over procedure if possible.

However the quorum is only used to detect the failure. In order to actually perform a failover, one of the Sentinels need to be elected leader for the failover and be authorized to proceed. This only happens with the vote of the majority of the Sentinel processes.

In this particular example, we're going to setup our nodes in a master/slave configuration, where we will have 1 master and 2 slave nodes. This way, if we lose one node, the cluster will still retain quorum and be able to elect a new master. In

this setup, writes will *have* to go through the master as slaves are read-only. The upside to this is that if the master disappears, its entire state has already been replicated to the slave nodes, meaning when one is elected as master, it can begin to accept writes immediately. This is different than setting up a redis cluster where data is sharded across master nodes rather than replicated entirely.

Since sentinel handles electing a master node and sentinel nodes communicate with each other, we can use it as a discovery mechanism to determine which node is the master and thus where we should send our writes.

Setup

To set up a cluster, we're going to run 3 redis instances:

- 1 master
- 2 slaves

Each of the three instances will also have a redis sentinel server running along side it for monitoring/service discovery. The config files I have for this example can be run on your localhost, or you can change the IP addresses to fit your own use-case. All of this will be done using version 3.0.2 of redis.

Configs

If you don't feel like writing configs by hand, you can clone the example repository I have at github.com/seanmcgary/redis-cluster-example. In there, you'll find a directory structure that looks like this:

```
redis-cluster
├── node1
│   ├── redis.conf
│   └── sentinel.conf
├── node2
│   ├── redis.conf
│   └── sentinel.conf
└── node3
    ├── redis.conf
    └── sentinel.conf
```

3 directories, 6 files

For the purpose of this demo, `node1` will be our starting master node and nodes 2 and 3 will be added as slaves.

Master node config

redis.conf

```
bind 127.0.0.1
port 6380

dir .
```

sentinel.conf

```
# Host and port we will listen for requests on
bind 127.0.0.1
port 16380

#
# "redis-cluster" is the name of our cluster
#
# each sentinel process is paired with a redis-server process
#
sentinel monitor redis-cluster 127.0.0.1 6380 2
sentinel down-after-milliseconds redis-cluster 5000
sentinel parallel-syncs redis-cluster 1
sentinel failover-timeout redis-cluster 10000
```

Our redis config should be pretty self-explanatory. For the sentinel config, we've chosen the redis-server port + 10000 to keep things somewhat consistent and make it easier to see which sentinel config goes with which server.

```
sentinel monitor redis-cluster 127.0.0.1 6380 2
```

The third "argument" here is the name of our cluster. Each sentinel server needs to have the same name and will point at the master node (rather than the redis-server it

shares a host with). The final argument (2 here) is how many sentinel nodes are required for quorum when it comes time to vote on a new master. Since we have 3 nodes, we're requiring a quorum of 2 sentinels, allowing us to lose up to one machine. If we had a cluster of 5 machines, which would allow us to lose 2 machines while still maintaining a majority of nodes participating in quorum.

```
sentinel down-after-milliseconds redis-cluster 5000
```

For this example, a machine will have to be unresponsive for 5 seconds before being classified as down thus triggering a vote to elect a new master node.

Slave node config

Our slave node configs don't look much different. This one happens to be for node2:

redis.conf

```
bind 127.0.0.1
port 6381

dir .

slaveof 127.0.0.1 6380
```

sentinel.conf

```
# Host and port we will listen for requests on
bind 127.0.0.1
port 16381

#
# "redis-cluster" is the name of our cluster
#
# each sentinel process is paired with a redis-server process
#
sentinel monitor redis-cluster 127.0.0.1 6380 2
sentinel down-after-milliseconds redis-cluster 5000
sentinel parallel-syncs redis-cluster 1
sentinel failover-timeout redis-cluster 10000
```

The only difference is this line in our `redis.conf`:

```
slaveof 127.0.0.1 6380
```

In order to bootstrap the cluster, we need to tell the slaves where to look for a master node. After the initial bootstrapping process, redis will actually take care of rewriting configs as we add/remove nodes. Since we're not really worrying about deploying this to a production environment where addresses might be dynamic, we're just going to hardcode our master node's IP address and port.

We're going to do the same for the slave sentinels as well as we want them to monitor our master node (`node1`).

Starting the cluster

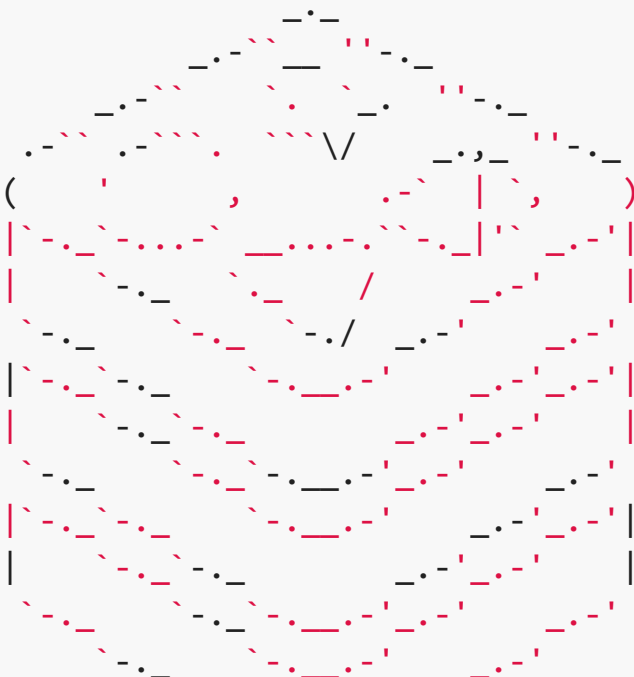
You'll probably want to run each of these in something like [screen](#) or [tmux](#) so that you can see the output from each node all at once.

Starting the master node

redis-server, node1

```
$ redis-server node1/redis.conf
```

```
57411:M 07 Jul 16:32:09.876 * Increased maximum number of open files to 1
```



Redis 3.0.2 (01888d1e/0) 64 bit

Running in standalone mode

Port: 6380

PID: 57411

<http://redis.io>

```

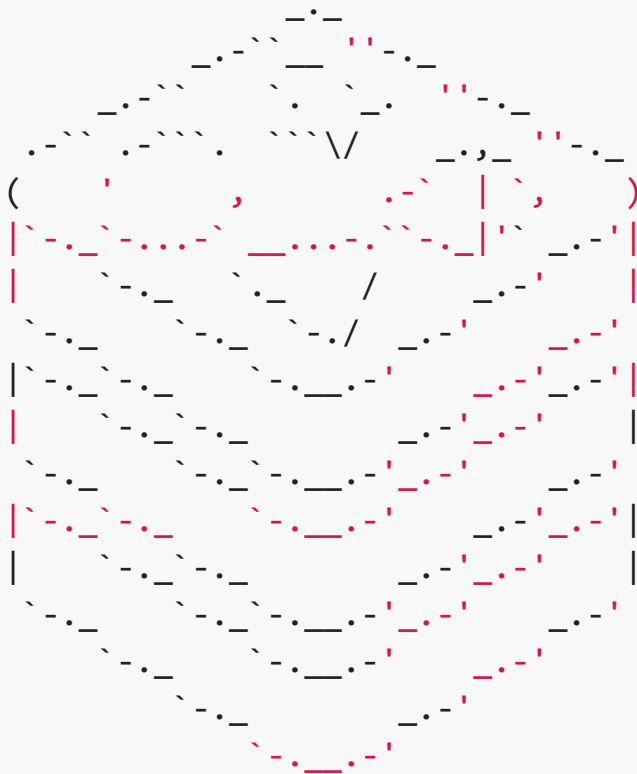
57411:M 07 Jul 16:32:09.878 # Server started, Redis version 3.0.2
57411:M 07 Jul 16:32:09.878 * DB loaded from disk: 0.000 seconds
57411:M 07 Jul 16:32:09.878 * The server is now ready to accept connections

```

sentinel, node1

```
$ redis-server node1/sentinel.conf --sentinel
```

```
57425:X 07 Jul 16:32:33.794 * Increased maximum number of open files to 1024
```



Redis 3.0.2 (01888d1e/0) 64 bit

Running in sentinel mode

Port: 16380

PID: 57425

<http://redis.io>

```
57425:X 07 Jul 16:32:33.795 # Sentinel runid is dde8956ca13c6b6d396d33e3a
```

```
57425:X 07 Jul 16:32:33.795 # +monitor master redis-cluster 127.0.0.1 6379
```

Starting the slave nodes

Now we can go ahead and start our slave nodes. As you start them, you'll see the master node report as they come online and join.

redis-server, node2

```
7.969 * Increased maximum number of open files to 10032 (it was originally
```

```

  _.-.-.-.-
-.-.-.-.-) Running in standalone mode
-.-.-.-.- Port: 6381
-.-.-.-.- PID: 57450
-.-.-.-.-

```

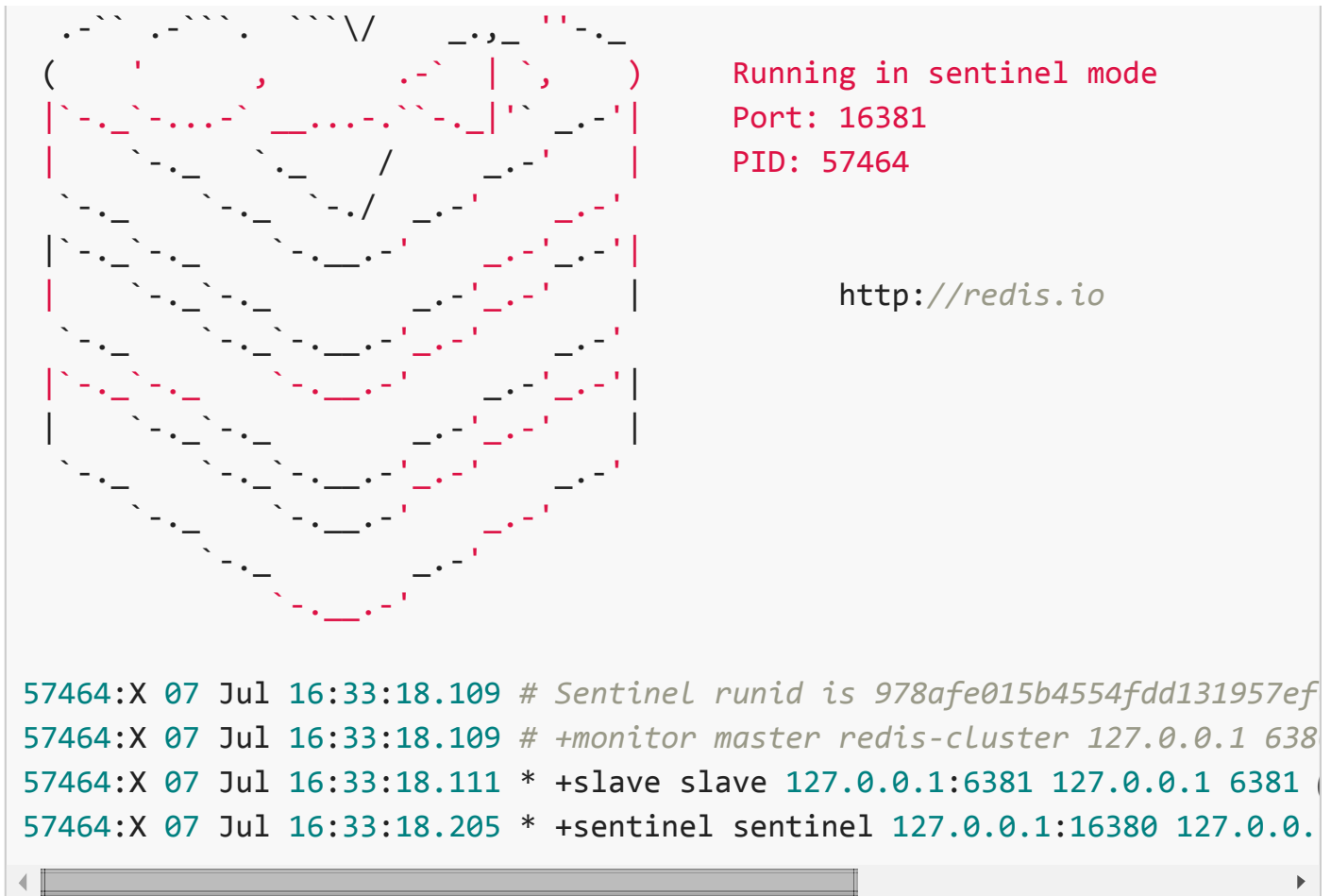
<http://redis.io>

```
7.971 # Server started, Redis version 3.0.2
7.971 * DB loaded from disk: 0.000 seconds
7.971 * The server is now ready to accept connections on port 6381
7.971 * Connecting to MASTER 127.0.0.1:6380
7.971 * MASTER <-> SLAVE sync started
7.971 * Non blocking connect for SYNC fired the event.
7.971 * Master replied to PING, replication can continue...
7.971 * Partial resynchronization not possible (no cached master)
7.971 * Full resync from master: d75bba9a2f3c5a6e2e4e9dfd70ddb0c2d4e647fd:
8.038 * MASTER <-> SLAVE sync: receiving 18 bytes from master
8.038 * MASTER <-> SLAVE sync: Flushing old data
8.038 * MASTER <-> SLAVE sync: Loading DB in memory
8.038 * MASTER <-> SLAVE sync: Finished with success
```

sentinel, node2

```
$ redis-server node2/sentinel.conf --sentinel
```

```
Redis 3.0.2 (01888d1e/0) 64 bit
```



```

Running in sentinel mode
Port: 16381
PID: 57464

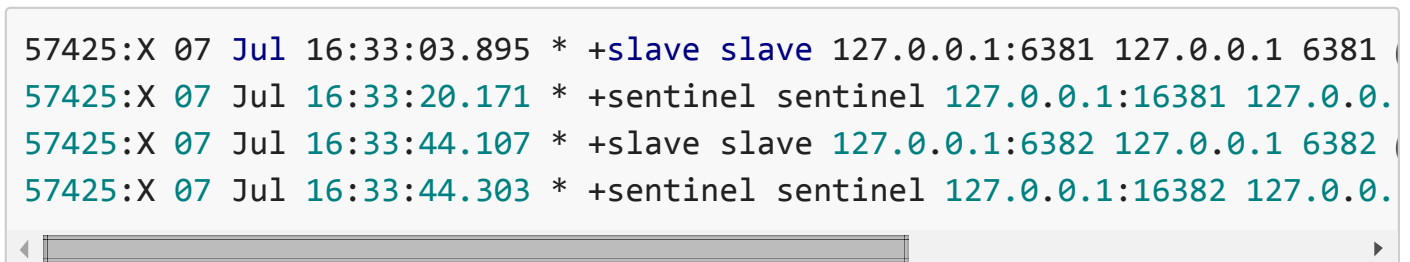
http://redis.io

57464:X 07 Jul 16:33:18.109 # Sentinel runid is 978afe015b4554fdd131957ef
57464:X 07 Jul 16:33:18.109 # +monitor master redis-cluster 127.0.0.1 6381
57464:X 07 Jul 16:33:18.111 * +slave slave 127.0.0.1:6381 127.0.0.1 6381
57464:X 07 Jul 16:33:18.205 * +sentinel sentinel 127.0.0.1:16380 127.0.0.1

```

Go ahead and do the same for `node3`.

If we look at the log output for `node1`'s sentinel, we can see that the slaves have been added:



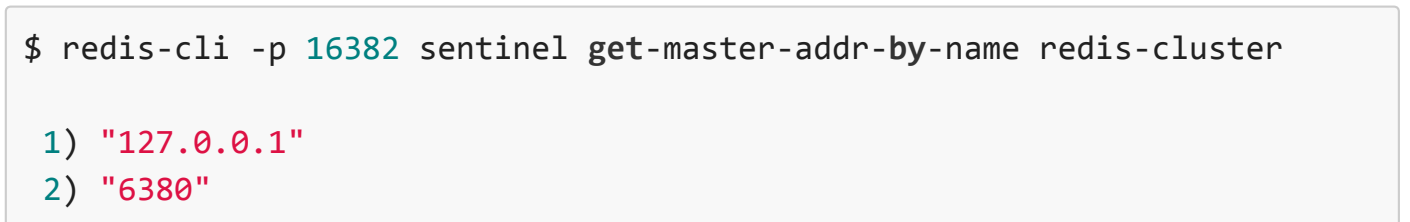
```

57425:X 07 Jul 16:33:03.895 * +slave slave 127.0.0.1:6381 127.0.0.1 6381
57425:X 07 Jul 16:33:20.171 * +sentinel sentinel 127.0.0.1:16381 127.0.0.1
57425:X 07 Jul 16:33:44.107 * +slave slave 127.0.0.1:6382 127.0.0.1 6382
57425:X 07 Jul 16:33:44.303 * +sentinel sentinel 127.0.0.1:16382 127.0.0.1

```

Find the master node

Now that our cluster is in place, we can ask sentinel which node is currently set as the master. To illustrate this, we'll ask sentinel on `node3`:



```

$ redis-cli -p 16382 sentinel get-master-addr-by-name redis-cluster

1) "127.0.0.1"
2) "6380"

```


As we can see here, the `ip` and `port` values match our `node1` which is our master node that we started.

Electing a new master

Now lets kill off our original master node

```
$ redis-cli -p 6380 debug segfault
```

Looking at the logs from `node2`'s sentinel we can watch the new master election happen:

```
270 # +sdown master redis-cluster 127.0.0.1 6380
301 # +new-epoch 1
301 # +vote-for-leader 2a4d7647d2e995bd7315d8358efbd336d7fc79ad 1
330 # +odown master redis-cluster 127.0.0.1 6380 #quorum 3/2
330 # Next failover delay: I will not start a failover before Tue Jul 7 1
432 # +config-update-from sentinel 127.0.0.1:16382 127.0.0.1 16382 @ redis
432 # +switch-master redis-cluster 127.0.0.1 6380 127.0.0.1 6381
432 * +slave slave 127.0.0.1:6382 127.0.0.1 6382 @ redis-cluster 127.0.0.1
432 * +slave slave 127.0.0.1:6380 127.0.0.1 6380 @ redis-cluster 127.0.0.1
519 # +sdown slave 127.0.0.1:6380 127.0.0.1 6380 @ redis-cluster 127.0.0.1
```

Now, lets see which machine got elected:

```
$ redis-cli -p 16382 sentinel get-master-addr-by-name redis-cluster
```

```
1) "127.0.0.1"
2) "6381"
```

Here, we can see that `node2` has been elected the new master of the cluster. Now, we can restart `node1` and you'll see it come back up as a slave since `node2` has been elected as the master node.

```
$ redis-server node1/redis.conf
```

```
57531:M 07 Jul 16:37:24.176 # Server started, Redis version 3.0.2
57531:M 07 Jul 16:37:24.176 * DB loaded from disk: 0.000 seconds
57531:M 07 Jul 16:37:24.176 * The server is now ready to accept connectio
```

```
57531:S 07 Jul 16:37:34.215 * SLAVE OF 127.0.0.1:6381 enabled (user request)
57531:S 07 Jul 16:37:34.215 # CONFIG REWRITE executed with success.
57531:S 07 Jul 16:37:34.264 * Connecting to MASTER 127.0.0.1:6381
57531:S 07 Jul 16:37:34.264 * MASTER <-> SLAVE sync started
57531:S 07 Jul 16:37:34.265 * Non blocking connect for SYNC fired the event
57531:S 07 Jul 16:37:34.265 * Master replied to PING, replication can continue
57531:S 07 Jul 16:37:34.265 * Partial resynchronization not possible (no partial syncs allowed)
57531:S 07 Jul 16:37:34.265 * Full resync from master: 135e2c6ec93d33dceb
57531:S 07 Jul 16:37:34.276 * MASTER <-> SLAVE sync: receiving 18 bytes from master
57531:S 07 Jul 16:37:34.276 * MASTER <-> SLAVE sync: Flushing old data
57531:S 07 Jul 16:37:34.276 * MASTER <-> SLAVE sync: Loading DB in memory
57531:S 07 Jul 16:37:34.276 * MASTER <-> SLAVE sync: Finished with success
```

That's it! This was a pretty simple example and is meant to introduce how you can setup a redis replica cluster with failover. In a followup post, I'll show how you can implement this on an actual cluster with CoreOS, containers, and HAProxy for loadbalancing.

🏷️ [coreos](#), [haproxy](#), [cluster](#), [redis](#), [sentinel](#), [redis-server](#), [high availability](#)

Share this post

You can edit before sharing



Twitter



Facebook



Email

Get blog posts in your inbox

Did you like what you read? Subscribe and we'll deliver new blog posts directly to your inbox.

SUBSCRIBE



Start the discussion...

Be the first to comment.

ALSO ON SEAN MCGARY.COM

WHAT'S THIS?

Cortex - express style routing for Backbone

3 comments • a year ago



Terry Cai — Great! This which I want to find. Backbone route need the middleware and queue control.

nsenter a systemd-nspawn container

1 comment • a year ago



Felipe — Hi Sean, Interesting series, I want to do some experiments in this area as well (running docker ...

Deploying Docker Containers on CoreOS Using Fleet

3 comments • 2 years ago



Benjamin Campbell — Fleet will also use the FLEETCTL_TUNNEL enviroment variable to establish an ...

Getting started with DeepLinkr

1 comment • 2 years ago



Wes Boudville — Sean, What happened to the deeplinkr.com website? I went there and it was ...